

Clean Architecture

Gregory DARDENNE

25 février 2026

Table des matières

1	Introduction	3
2	Problématique des architectures classiques	3
3	Principes de la Clean Architecture	3
3.1	Indépendance des frameworks	4
3.2	Indépendance de la base de données	4
3.3	Indépendance de l’User Interface (UI)	4
4	Structure des couches	5
4.1	Domain	5
4.1.1	Entities	5
4.1.2	Exceptions	6
4.1.3	Enums	6
4.1.4	Repositories	6
4.2	Application	6
4.2.1	Data Transfer Objects (DTOs)	6
4.2.2	Services	6
4.2.3	Queries/Commands	6
4.3	Infrastructure	7
4.3.1	Repositories	7
4.3.2	Services	7
4.4	Presentation	7
4.4.1	Controllors	7

4.4.2	Views	7
5	Exemple concret en C#	8
5.1	Structure en code	8
5.2	Domain	9
5.2.1	Entities	9
5.2.2	Exceptions	9
5.2.3	Repositories	9
5.3	Application	9
5.4	Infrastructure	9
5.5	Presentation	9
6	Injection de dépendances	9
7	Avantages et inconvénients	9
7.1	Avantages	9
7.2	Inconvénients	10
8	Conclusion	10

1 Introduction

Ce document vise à expliquer ce qu'est la Clean Architecture. Il présente les problématiques rencontrées dans les architectures classiques et montre que la Clean Architecture n'est pas une lubie de développeur devenu excessivement théorique, mais une réponse structurée à des enjeux réels.

L'objectif est qu'à l'issue de cette lecture, chacun comprenne les enjeux architecturaux et les bénéfices d'une telle approche. La mise en œuvre d'une Clean Architecture n'est pas triviale, en particulier dans un projet existant. Sans une compréhension partagée par l'ensemble de l'équipe, son adoption peut être compromise, notamment en raison de certains inconvénients initiaux pouvant paraître dissuasifs.

2 Problématique des architectures classiques

Dans les architectures traditionnelles, on finit souvent par rencontrer des freins qui ralentissent l'évolution du projet. Ces blocages peuvent affecter la stabilité, la testabilité et la scalabilité de l'application.

Dans ces architectures, le code dépend fortement des frameworks, de la base de données ou d'outils techniques spécifiques. Cette dépendance entraîne un couplage fort entre le cœur métier et les éléments externes.

Une application dépendante de sa base de données devient difficile à tester sans infrastructure réelle. De même, une application fortement liée à un framework peut rencontrer des difficultés lors des montées de version ou des changements technologiques.

Le noyau métier de l'application dépend alors de l'extérieur, ce qui va à l'encontre des principes de conception robustes et évolutifs.

3 Principes de la Clean Architecture

La Clean Architecture repose sur un principe fondamental : inverser les dépendances afin que le noyau de l'application ne dépende d'aucune technologie externe.

On peut la visualiser comme un oignon composé de plusieurs couches concentriques. Ce modèle est d'ailleurs inspiré de l'architecture en oignon (Onion Architecture).

Le principe central est le suivant : toutes les dépendances doivent pointer vers le noyau métier.

Ainsi, les règles métier restent indépendantes :

- des frameworks,
- de la base de données,
- de l'interface utilisateur,
- des outils externes.

3.1 Indépendance des frameworks

Dans une Clean Architecture, le cœur de l'application ne dépend d'aucun framework spécifique. Les règles métier ne doivent pas connaître ASP.NET Core, Entity Framework ou tout autre outil technique.

Le framework devient un détail d'implémentation et non un élément structurant de l'architecture. Cela signifie que les entités et les cas d'usage doivent pouvoir être exécutés et testés sans infrastructure web.

Concrètement, un Use Case ne doit pas dépendre d'un controller ASP.NET, mais d'interfaces définies dans la couche Application. L'implémentation concrète, elle, sera fournie dans la couche Infrastructure.

3.2 Indépendance de la base de données

Dans une architecture classique, les entités métier sont souvent directement liées à la base de données via un Object-relational Mapping (ORM) comme Entity Framework. Cela crée une dépendance forte entre le modèle métier et la technologie de persistance.

Dans une Clean Architecture, la base de données est considérée comme un détail. Le cœur métier ne dépend pas d'Entity Framework ni d'un moteur SQL spécifique.

Les interfaces de repository sont définies dans la couche Application ou Domain, et leur implémentation concrète est réalisée dans la couche Infrastructure.

Ainsi, il serait possible de remplacer une base SQL par une base NoSQL, ou même par un stockage en mémoire, sans modifier les règles métier.

3.3 Indépendance de l'UI

L'interface utilisateur ne doit pas contenir de logique métier. Son rôle est uniquement de transmettre des données aux cas d'usage et d'afficher les résultats.

Dans une Clean Architecture, l'UI dépend des couches internes, mais l'inverse est interdit. Cela permet de changer l'interface sans impacter le cœur métier.

Par exemple, une application pourrait proposer successivement :

- une Application Programming Interface (API) REST,
- une interface web,
- une application mobile,
- une interface en ligne de commande.

Toutes ces interfaces pourraient utiliser les mêmes cas d'usage sans modification du noyau applicatif.

4 Structure des couches

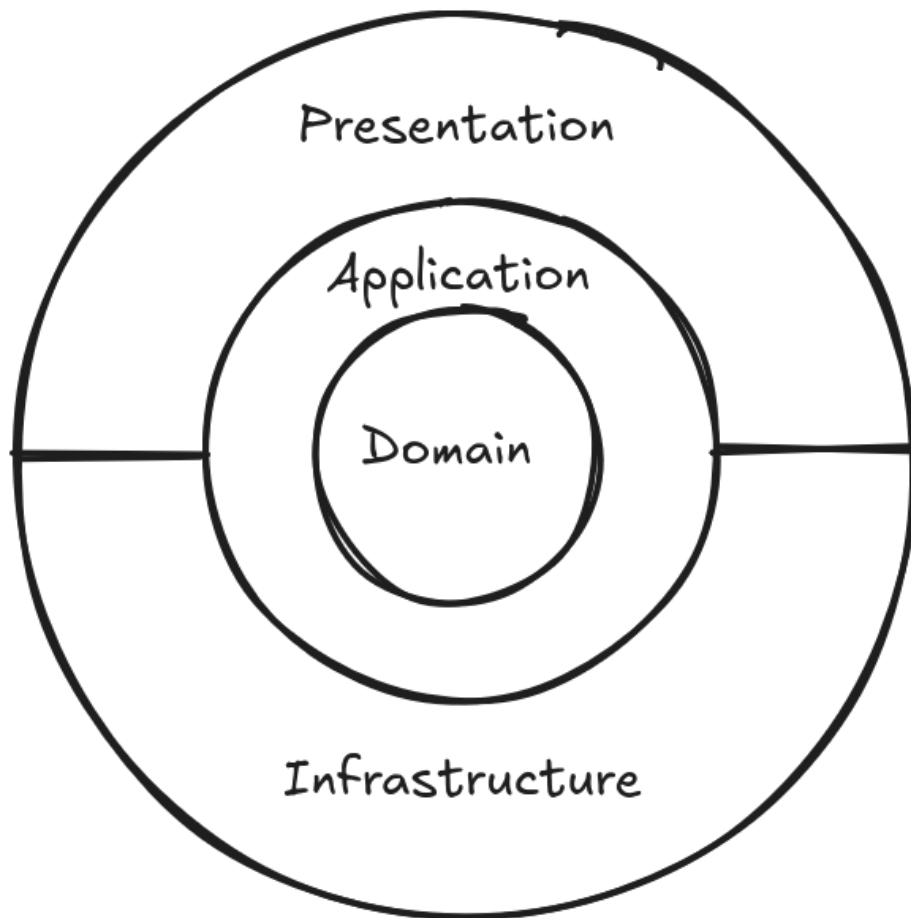


FIGURE 1 Schéma de la Clean Architecture

4.1 Domain

Le domaine constitue le cœur de l'application. C'est dans cette couche que sont déclarées les entités métier, ainsi que les différentes interfaces qui serviront de contrats entre les couches de l'architecture.

4.1.1 Entities

Une entité désigne un objet ou un concept du monde. Il est souvent persisté dans des systèmes spécialisés.

Il est le cœur métier de notre Application et contient toujours un ID pour les différencier les uns des autres.

On peut prendre l'exemple d'un utilisateur dans une application. Un ID unique le différencie des autres et d'autres informations comme son nom, son prénom, sa date de naissance permettent de le définir.

4.1.2 Exceptions

Permet d'avoir des exceptions personnalisé à notre application. Au lieu d'utiliser des exceptions classiques il est préférable de faire nos propres exceptions à partir de celle existante.

4.1.3 Enums

Les énumérations sont des fichiers permettant de représenter une liste. Par exemple une liste de statut utilisateur (en ligne, en attente, hors ligne, occupé)

4.1.4 Repositories

Les repositories dans ce contexte ne concerne pas leur implémentation. Dans le domain on déclare uniquement les interfaces des repositories

4.2 Application

La couche application orchestre la logique métier.

4.2.1 DTOs

Les DTOs permettent de transférer des données spécifiques entre la couche application et les couches supérieur (Presentation/Infrastructure)

4.2.2 Services

L'application doit déclarer les interfaces pour les différents services dont elle a besoin.

Aucune implémentation ne doit être fait dans ce dossier.

4.2.3 Queries/Commands

Les Queries et Commands sont des use cases. Il permet de définir des suites d'actions suite à un événement. Il est l'orchestrateur des repositories et des services et permet de retourner des informations.

Les Queries sont le termes pour les use cases qui demande que de la lecture¹.

Les Commands sont le termes pour les use cases qui demande que de l'écriture².

1. Récupération dans la base de données pour un affichage

2. Modification d'un élément dans la base de données

4.3 Infrastructure

La couche infrastructure constitue le socle technique de l'application. On y retrouve l'implémentation des accès aux base de données, les appels à des services externes ou encore la gestion des communications réseau. Étant une des couches les plus haute de l'application, la présence d'interface à cette étage n'est pas cohérent

4.3.1 Repositories

L'implémentation des repositories déclaré dans le Domain, se fait ici. Ainsi, l'infrastructure n'interagi pas avec l'application.. Et l'infrastructure n'est qu'une implémentation de la persistance qu'on peut faire bien des manières. Aucun repository ne doit se retrouver sans interface

4.3.2 Services

On implémente les services déclarés dans la couche applicative. Chaque service doit dépendre d'une interface.

4.4 Presentation

La couche présentation est le point d'entrée de l'application. Elle est responsable de l'exposition des données vers l'extérieur, que ce soit au travers de contrôleurs API, de vues ou de tout autre mécanisme permettant à l'utilisateur ou à des systèmes tiers d'interagir avec l'application.

4.4.1 Controllers

Les controllers dans cette structure ne sont que des appels. Il reçoit une requête, et la transmet direction à un use case. Dans le cas MVC vu qu'on retourne des views, il utilise le DTO qu'il reçoit du use case pour créer une view.

4.4.2 Views

Les views doit uniquement prendre des DTO en entrée. On s'interdit l'interaction avec les différentes entités.

5 Exemple concret en C#

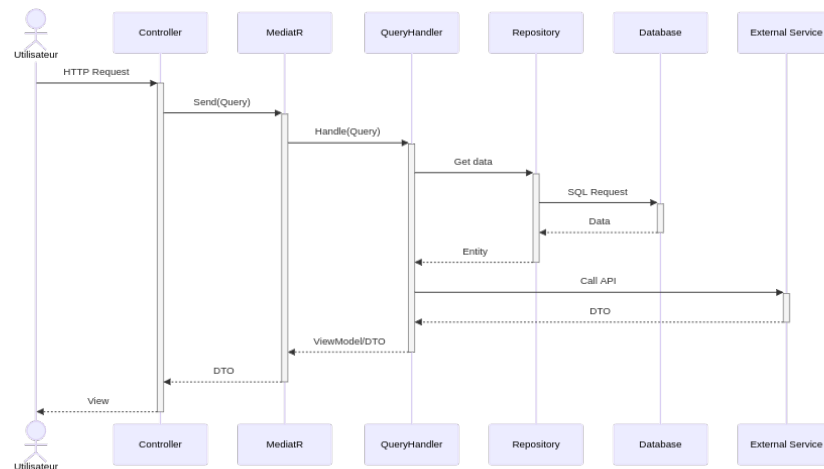


FIGURE 2 Schéma d'une action utilisateur

Un utilisateur arrive sur votre site et veut afficher une page.

Une requête HTTP se génère et le controller dans votre application l'intercepte.

Le controller utilise le service MediatR pour rediriger cette requête vers un Handler.

Le handler va faire une suite d'action en commençant par récupérer des données en base de données grâce aux repositories. Puis fait par exemple un appel à un service d'identification. Ce dernier renvoie des informations et le handler retourne un DTO prêt pour l'affichage.

Le controller reçoit le DTO et le transmet à une View qui va permettre à l'utilisateur de voir sa page qu'il vient de demander.

5.1 Structure en code

```

project/
|-- project.Application
|   '-- User
|       |-- CreateUser
|       |   |-- CreateUserCommand.cs
|       |   |-- CreateUserHandler.cs
|       |   |-- CreateUserRequest.cs
|       |   '-- CreateUserResponse.cs
|       '-- Interfaces
|           '-- IEmailService.cs
|-- project.Domain
|   |-- Entities
|   |   '-- User.cs
|   |-- Enums

```



```
| | '-- status.cs
| |-- Exceptions
| | '-- TuEnvoieLeMauvaisDtoException.cs
| '-- Repositories
|     '-- IUserRepository.cs
|-- project.Infrastructure
|   |-- Persistance
|   |   |-- AppDbContext.cs
|   |   '-- Configurations
|   |-- Repositories
|   |   '-- UserRepository.cs
|   '-- Services
|       '-- EmailService.cs
'-- project.Presentation
    |-- Controllers
    |   '-- UserController.cs
    '-- Views
        '-- User
            '-- PageParametre.cshtml
```

5.2 Domain

5.2.1 Entities

5.2.2 Exceptions

5.2.3 Repositories

5.3 Application

5.4 Infrastructure

5.5 Presentation

6 Injection de dépendances

7 Avantages et inconvénients

7.1 Avantages

La Clean Architecture a de nombreux avantages.

Le premier est la séparation claire des responsabilités, elle permet d'avoir un code plus clair et de se retrouver plus facilement dans ce dernier.

La maintenabilité, le code devient beaucoup plus maintenable car tous les éléments ont leur propre responsabilité ainsi si une action plante, il est facile de retrouver le responsable.

Les tests unitaires, la séparation permet d'avoir un environnement isolé ce qui permet de créer des tests plus facilement.

On obtiens aussi une indépendance des externes, une base de données n'est plus un requis pour l'application mais juste un outil de stockage. Ce qui permet de changer, remplacer adapter plus facilement nos outils externes.

7.2 Inconvénients

Cependant, cette architecture amène énormément de complexités dans la structure du projet. Les développeurs pas habitués à ce genre d'architecture se retrouvent facilement perdus, demandant ainsi un bon suivi par les développeurs plus habitués. Il demande une bonne compréhension de la structure au risque de mélanger les notions.

8 Conclusion

En conclusion, la clean architecture n'est pas une solution universelle et son adoption doit être mûrement réfléchie. Elle s'avère peu adaptée aux projets de petite envergure à faible évolution, où elle engendrerait une complexité structurelle injustifiée et une perte de temps considérable.

Cependant, notre contexte s'inscrit précisément dans le cas d'usage pour lequel cette architecture a été conçue. Le projet gagne en complexité et devient de plus en plus difficile à maintenir. Des problématiques concrètes telles que la mise en place de tests unitaires ou la migration de SQL Server vers PostgreSQL trouveraient une réponse naturelle et élégante dans ce type d'architecture.

Rejeter la clean architecture au moment même où le projet entre dans son cas d'usage idéal serait donc une décision contre-productive, privant l'équipe des bénéfices qu'elle est précisément censée apporter dans ce type de situation.

Glossaire

- controller** Utiliser pour parler des objets qui permettent de traiter une requête web. 4
- framework** Ensemble d'outils et de règles préétablis qui fournit une structure de base pour développer des applications de manière organisée et efficace. 1, 3, 4
- interface** Une interface est un fichier qui permet de définir des attentes (exemple: Nos attentes d'une voiture c'est qu'elle roule). 4-7
- repository** Un repository est un objet permettant de faire le lien entre une entité et une base de données. 1, 6-8
- service** Un service est un objet permettant d'implémenter des logiques complexes (services de mail, gestionnaire des sauvegarde de document, etc...). 1, 6, 7
- use case** Un use case ou cas d'usage provient des diagrammes de cas d'usage qui permet de définir une suite d'action possible à un utilisateur dans notre application. Chaque cas doit pouvoir être traduit en différent diagramme.. 6, 7

Acronymes

- API** Une interface accessible pour récupérer ou envoyer des informations. 4
- DTO** Un objet permettant de transférer des données aux travers des couches d'une application. 1, 6, 7
- ORM** Technique pour convertir les données entre une base de données et la mémoire. 4
- UI** Interface utilisateur. 1, 4

Références

DRUNKNCODE (2024). *Clean Architecture : Simplified and In-Depth Guide*. URL : <https://medium.com/@DrunknCode/clean-architecture-simplified-and-in-depth-guide-026333c54454>.

SALEEM, Muhammed (2024). *Clean Architecture in .NET*. URL : <https://code-maze.com/dotnet-clean-architecture>.